Overview: artificial neural networks

- **A neural network is a stack of simple units (neurons).** Each neuron takes a weighted sum of its inputs and passes it through a nonlinear function
- **Here it is used for supervised learning.** Same setup as before: features, a target, a loss, and training, validation, and test sets
- **What it adds:** it can learn a nonlinear relationship between many inputs and an output without you choosing the shape of the curve
- **What it costs:** thousands of weights to learn, so it needs enough data, careful validation, and regularization
- **This week:** the basics on handwritten digits, the standard first example, then the same ideas on climate data

Handwritten digits (MNIST)
70,000 images of the digits 0 to 9. Used to show what a network is and how it learns.

New York City electricity demand
Daily demand against temperature, 2019 to 2024. Used to show what a single neuron does.

Central Park temperature
The average seasonal cycle, 1990 to 2025. Used to show what more hidden neurons add.

Climate model parameter ensembles
Hundreds of runs of CAM6 and ModelE3, each with 45 uncertain parameters changed. The tutorial builds an emulator from them.

Neural network applications in climate science

Parameterizations
Replace a slow scheme for clouds or convection inside a climate model with a fast network trained on a high-resolution model.

Emulators
Predict a climate model's output from emissions or forcings in seconds, so many more settings can be explored.

Weather forecasting
Networks trained on decades of reanalysis now produce global medium-range forecasts.

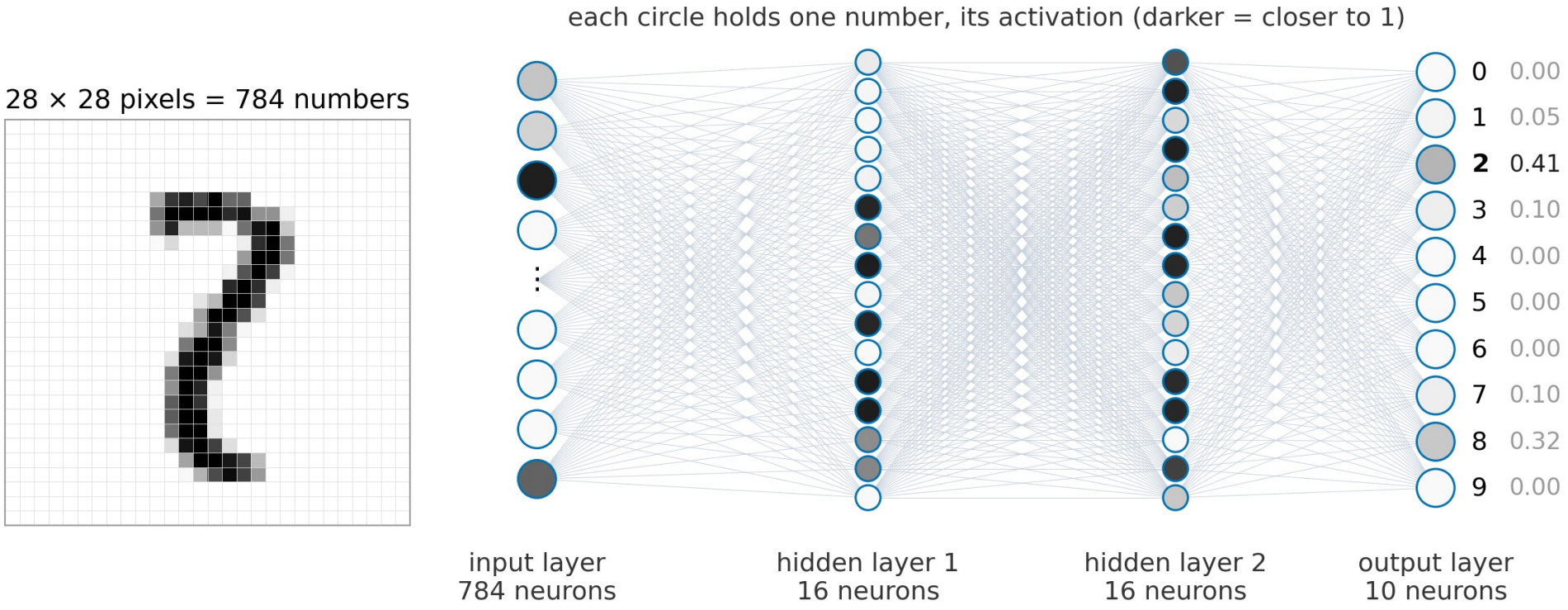
Downscaling and bias correction
Map coarse model output to local scales, or correct a model's systematic errors against observations.

Remote sensing
Retrieve surface or atmospheric properties from satellite images. Convolutional networks

Hydrology and energy
Streamflow from rainfall, electricity demand from weather, and wind and solar output from forecasts.

- **Common features:** a nonlinear relationship, many inputs, and enough data to learn from, whether observations, reanalysis, or model runs
- **Typical use:** the network replaces something slow, such as a physical model, or something hand-built, such as a regression with chosen terms

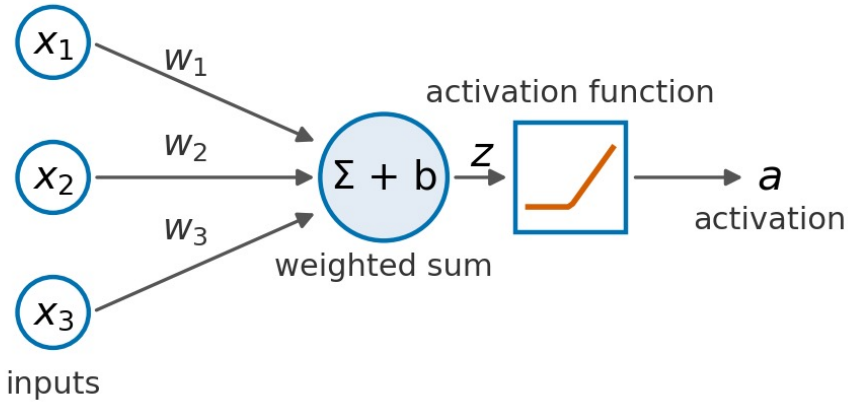
A neural network for handwritten digits



- **A neuron holds one number, its activation.** The 784 input neurons hold the pixel brightnesses. Each later neuron's number is computed from the layer before it
- **Layers:** 784 inputs, two hidden layers of 16 neurons, and 10 outputs, one per digit. The largest output is the network's answer
- **The whole network is a function:** 784 numbers in, 10 numbers out. Every line in the diagram is one weight

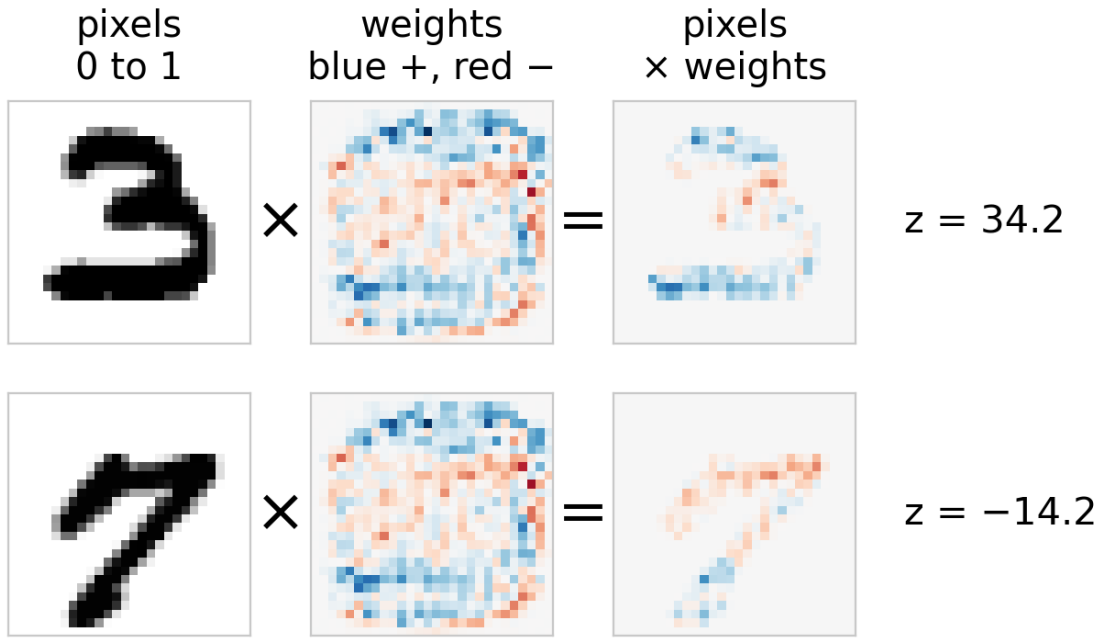
MNIST: 70,000 handwritten digits, 28 × 28 pixels. Following 3Blue1Brown's neural network lessons by Grant Sanderson, 3blue1brown.com/lessons/neural-networks. Network trained for these slides with scikit-learn.

Inside one neuron: weights, bias, and activation



$$z = w_1 x_1 + w_2 x_2 + \cdots + w_{784} x_{784} + b$$

$$a = \sigma(z)$$



Weight: how much an input matters, and in which direction.

Bias: shifts the switch-on point. The neuron turns on when the weighted inputs add up to more than −b.

Activation function σ: turns z into the neuron's activation a, the number it passes on. This network uses the sigmoid, which returns a number between 0 and 1.

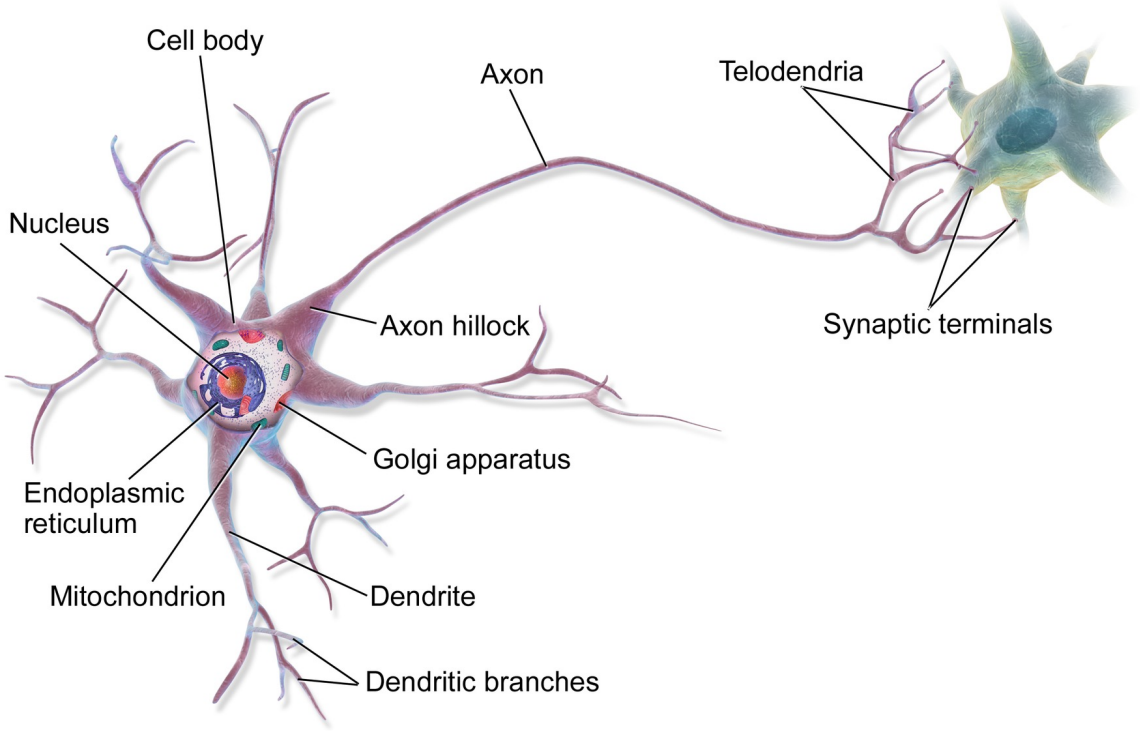
One neuron from hidden layer 1. Each pixel is multiplied by its weight, then the products and the bias b are added to give z.

The 3 falls mostly on blue weights: z = 34.2, so a = σ(z) ≈ 1 and the neuron is on.

The 7 falls mostly on red weights: z = −14.2, so a ≈ 0 and the neuron stays off.

Following 3Blue1Brown's neural network lessons by Grant Sanderson, 3blue1brown.com/lessons/neural-networks. Network trained for these slides with scikit-learn.

Biological neurons and artificial neurons

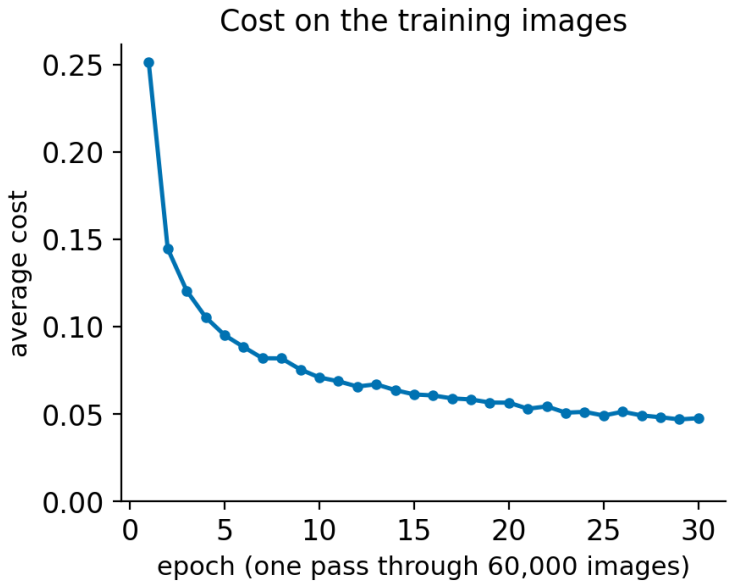


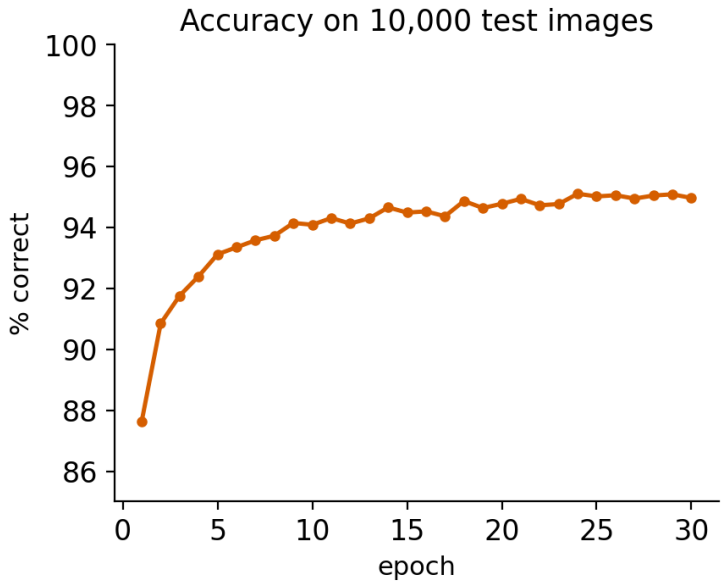
Real neuron	What it does	Artificial neuron
Dendrites	receive signals from other neurons	inputs $x_1 \dots x_n$
Synapses	strong or weak, exciting or inhibiting	weights w, positive or negative
Cell body	adds up the incoming signals	weighted sum z
Axon hillock	fires a pulse only if the total passes a threshold	bias b and activation function σ
Axon	carries the pulse to thousands of other neurons	activation a, an input to the next layer

- **Scale:** a human brain has about 86 billion neurons, each connected to thousands of others. The digit network has 42 neurons
- **Differences:** real neurons signal with timed electrical pulses, rewire themselves, and learn by rules still being studied. Artificial neurons pass numbers forward and learn by gradient descent

Neuron illustration: BruceBlaus (Blausen Medical), CC BY 3.0, via Wikimedia Commons. Comparison follows Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. (2022), chapter 10.

Learning: choosing the 13,002 weights and biases by minimizing cost function





$$\text{cost} = \sum_{k=0}^{9} (a_k - y_k)^2$$

a is the network's output for one image. y is 1 for the correct digit and 0 for the other nine.

- **Count:** 784 × 16 + 16 × 16 + 16 × 10 weights, plus 16 + 16 + 10 biases, is 13,002 numbers
- **Training:** average the cost over the 60,000 training images, then nudge all 13,002 numbers in the direction that lowers it fastest. That is gradient descent
- **Result:** 87.6% of the test images right after one pass through the training set, 95.0% after 30

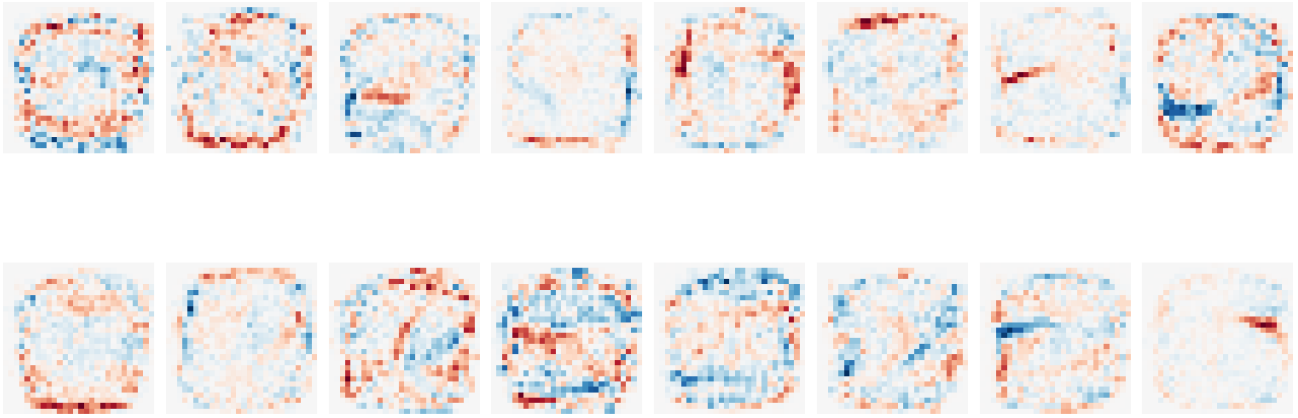
Following 3Blue1Brown's neural network lessons by Grant Sanderson, 3blue1brown.com/lessons/neural-networks. Network trained for these slides with scikit-learn.

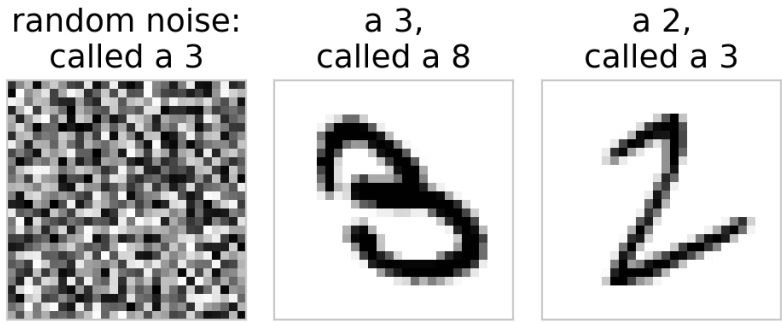
What the trained digit network learned

784 pixels → edges → loops and lines → a digit

What we might hope each layer finds

weights of the 16 neurons in hidden layer 1 (blue +, red −)

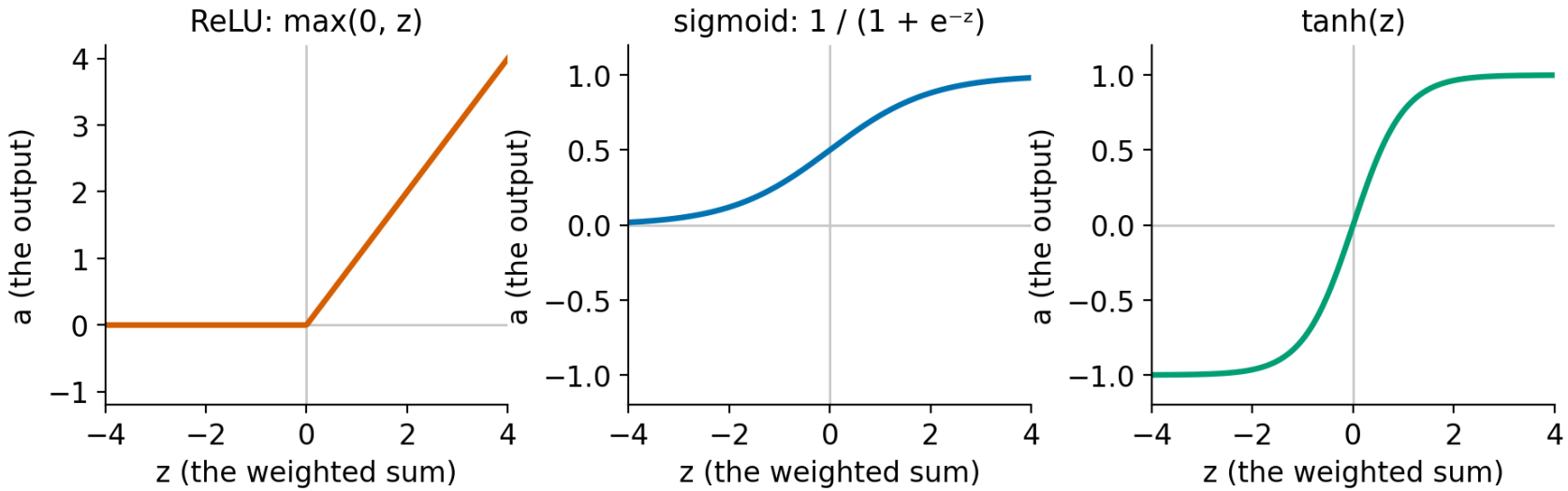




- **The weights are mostly noisy patterns with a few strokes,** not clean edges. The network found a way to get 95.0% right, but not the way we imagined
- **It always answers with a digit.** It calls 90% of random noise images a 3, with an average confidence of 0.90. It has no way to say 'not a digit'
- **For climate data:** a good score does not mean the network learned the physics, and it can answer confidently on inputs unlike its training data

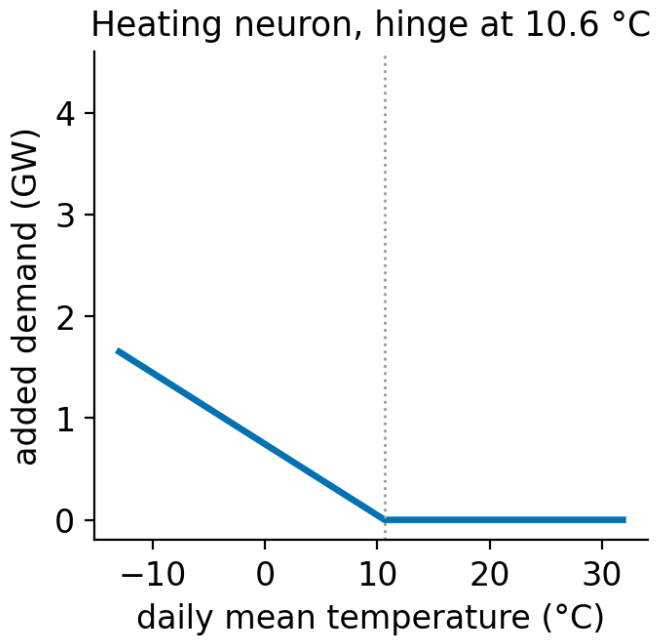
Following 3Blue1Brown's neural network lessons by Grant Sanderson, 3blue1brown.com/lessons/neural-networks. Network trained for these slides with scikit-learn. 503 of the 10,000 test images are misclassified.

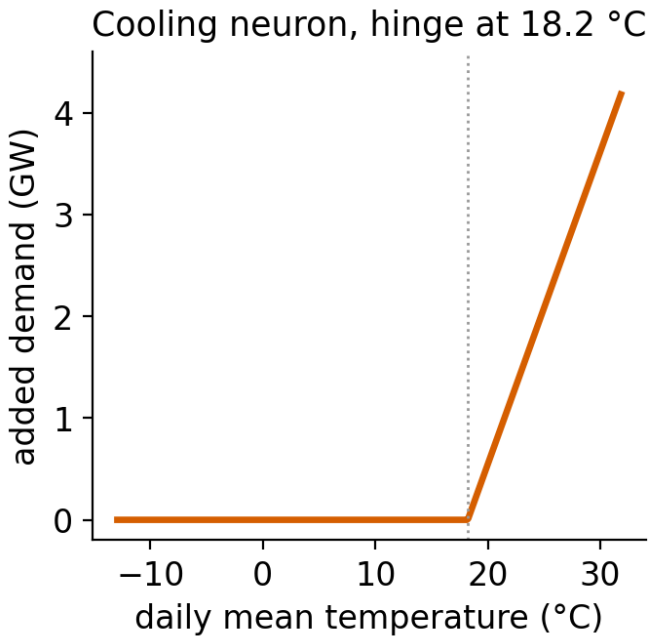
Activation functions and why they are needed

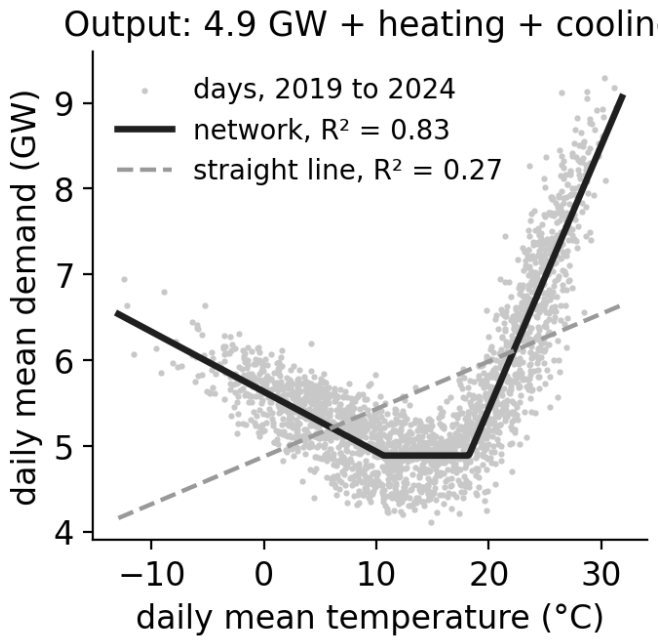


- **Without an activation function in the hidden layers, depth adds nothing.** A weighted sum of weighted sums is still a weighted sum, so any stack of linear layers is one linear model
- **ReLU:** the default for hidden layers. Cheap to compute, and its slope is 1 for positive inputs, which helps deep networks train
- **Sigmoid and tanh:** smooth and bounded, 0 to 1 and −1 to 1. The digit network uses the sigmoid

One hidden layer of two neurons fits NYC demand



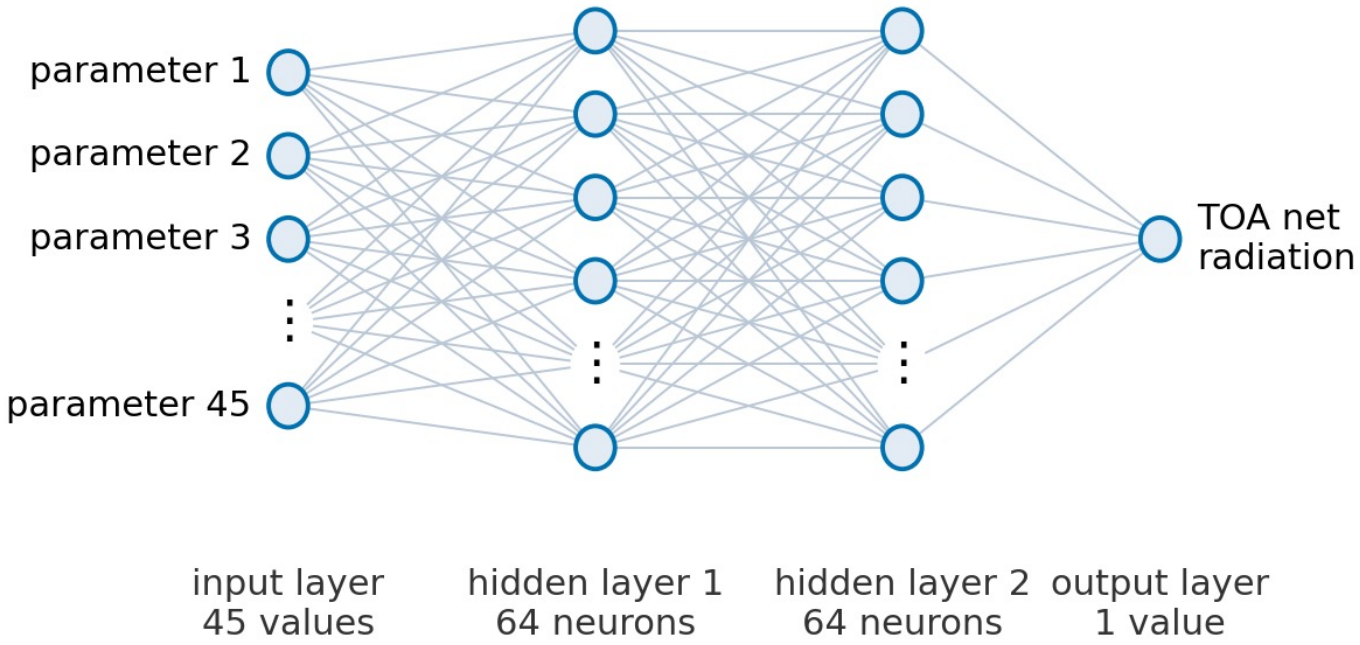




- **The network:** one input (temperature), one hidden layer of two ReLU neurons, and an output that adds them to a constant. Seven parameters in total
- **What it learned:** demand rises below 10.6 °C (heating) and above 18.2 °C (cooling). The cooling threshold is close to the 18.3 °C used for degree days, and the network was never given it
- **Fit:** R^2 = 0.83, against 0.27 for a straight line. A network with 16 neurons scores 0.83, so two are enough here

Daily mean electricity demand in New York City (NYISO) and temperature, 2,186 days from 2019 to 2024. scikit-learn MLPRegressor, standardized temperature.

From one neuron to a multilayer perceptron



Multilayer perceptron (MLP): layers of neurons, each layer feeding the next, from the inputs to the output.

Dense, or fully connected, layer: every neuron sees every value from the layer before, each with its own weight. Keras calls a neuron a unit.

$$a_1 = \varphi(W_1 x + b_1)$$

$$a_2 = \varphi(W_2 a_1 + b_2)$$

$$\hat{y} = W_3 a_2 + b_3$$

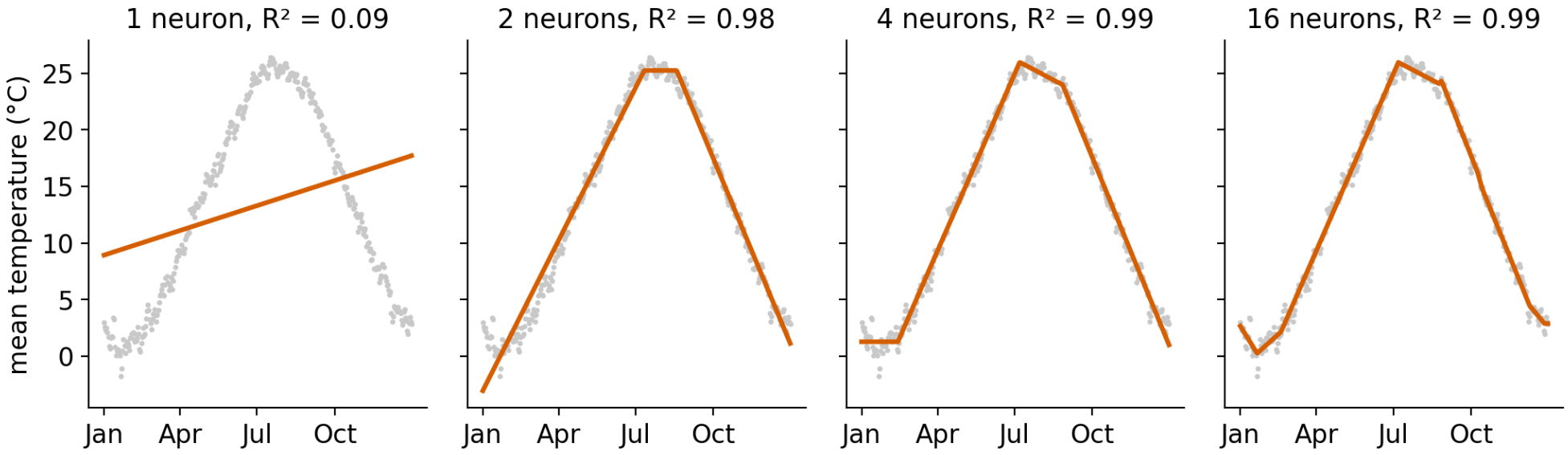
φ**:** the activation function, applied to each neuron's weighted sum. ReLU in this network.

Hidden layers: the layers between input and output. Their outputs a_1 and a_2 are features the network builds for itself, and they rarely have a physical meaning.

Deep learning: networks with several hidden layers.

The network in this week's tutorial: 45 parameters of the CAM6 climate model in, global top-of-atmosphere net radiation out. Only a few neurons in each layer are drawn.

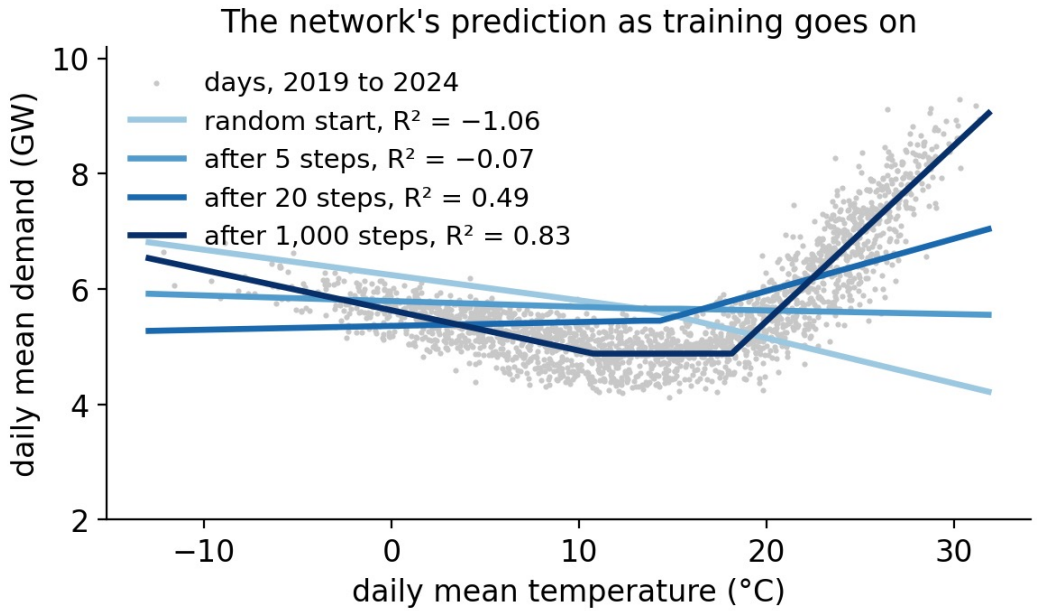
More hidden neurons allow more bends

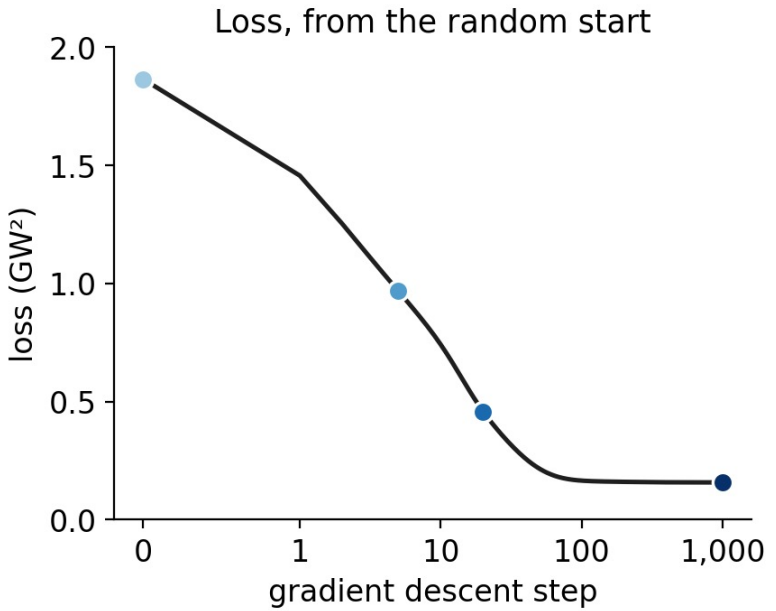


- **Each ReLU neuron adds one bend to the output.** One neuron gives one hinge, which cannot make a peak. Two neurons make a peak, and more neurons round it off
- **Universal approximation theorem:** one hidden layer with enough neurons can approximate any continuous function as closely as you like (Cybenko 1989, Hornik et al. 1989)
- **What the theorem does not say:** how many neurons you need, how much data, or whether training will find the right weights
- **In practice,** several hidden layers of moderate width usually train more easily than one very wide layer

Central Park daily mean temperature averaged for each day of the year, 1990 to 2025 (NOAA daily summaries). Networks with one hidden layer of 1, 2, 4, and 16 ReLU neurons.

Training starts from random weights

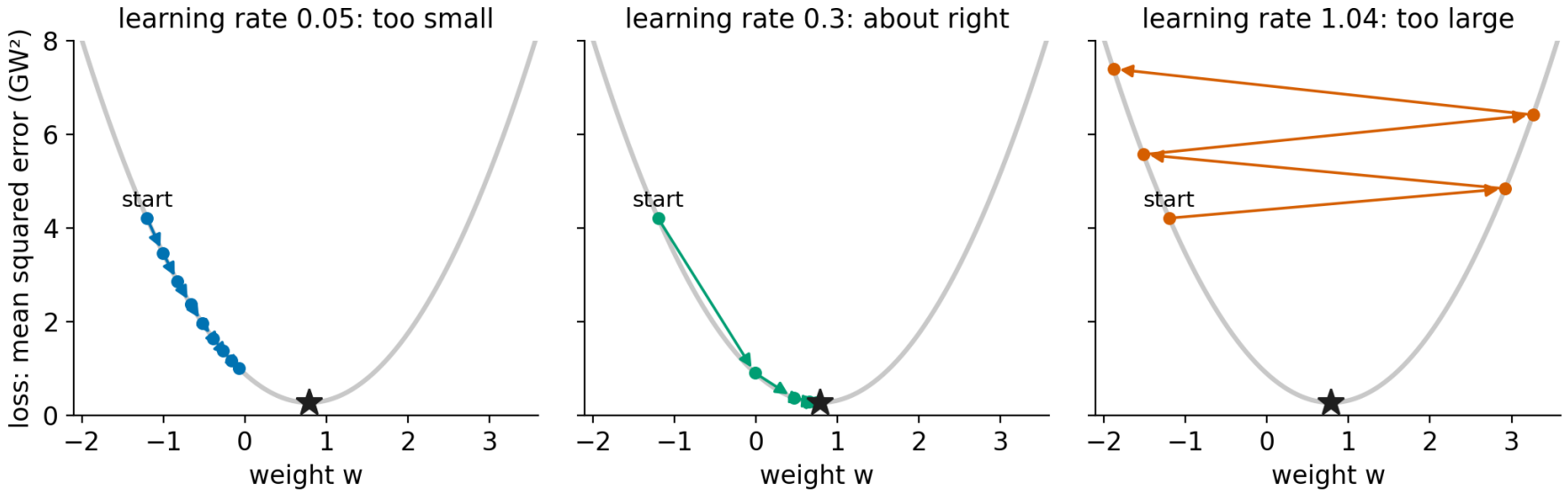




- **Start:** every weight is a small random number and every bias is 0, the Keras default. The first prediction is poor. This start has demand falling on hot days, $R^2 = -1.06$
- **Not all zeros:** neurons that start identical get identical gradients and stay identical, so the layer would act like one neuron
- **Then step downhill, over and over:** each step uses the slope of the loss at the current weights. After 1,000 steps this start reaches the heating and cooling fit, $R^2 = 0.83$

The two-neuron network for New York City daily demand, 2,186 days. One random start, then plain gradient descent using all days at every step, learning rate 0.1, temperature and demand standardized.

Training: gradient descent and the learning rate



$$L(w) = \frac{1}{n}\sum_i (wx_i + b - y_i)^2$$

$$\frac{\partial L}{\partial w} = \frac{2}{n}\sum_i (wx_i + b - y_i)x_i$$

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

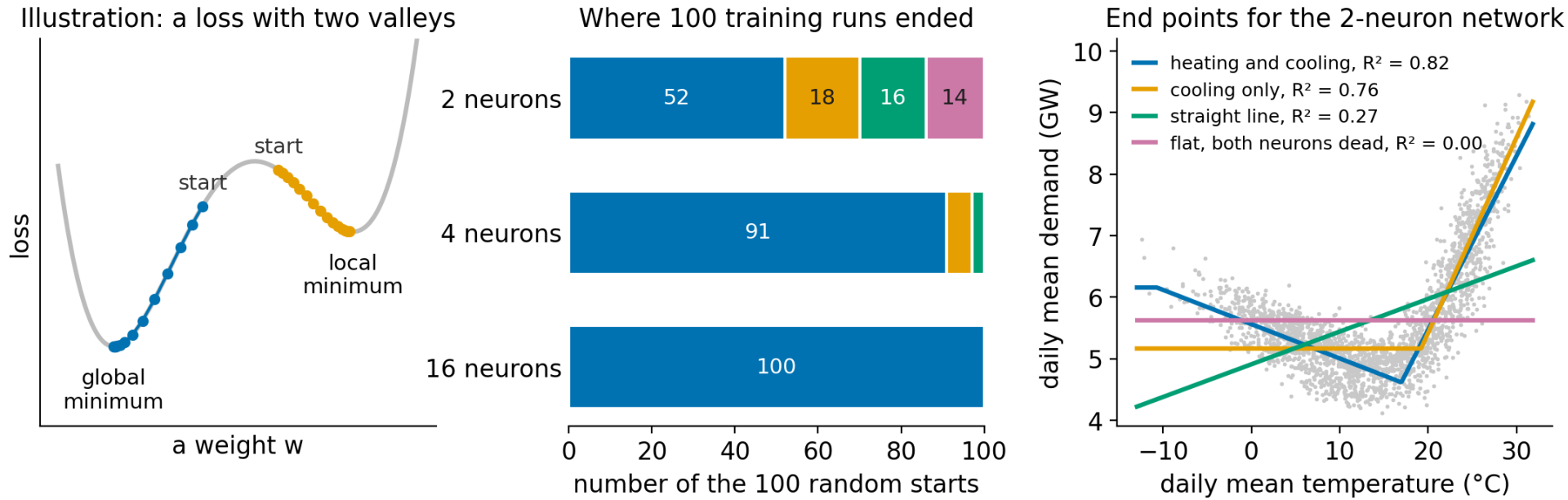
Loss L: the mean squared error over the n = 680 days. x_i is the standardized temperature and y_i the demand on day i.

Gradient ∂L/∂w: the slope of the loss at the current weight. Differentiating the formula for L gives it exactly, from that one point. Step downhill, then repeat.

Learning rate η: the size of each step. Too small is slow, too large overshoots and the loss grows. 0.001 is a common starting value.

Fitting demand = w × temperature + b to the 680 New York days above 20 °C, with temperature standardized and b fixed at its best value, so the loss depends on one weight.

When the loss has more than one minimum



- **A network's loss has many low points.** Linear regression's loss is one bowl with one minimum. A network's loss also has local minima and flat stretches where the gradient is near zero. Gradient descent stops in whichever valley it reaches
- **Same data, same network, 100 random starts:** with 2 neurons, 52 runs found heating and cooling. The rest stopped at cooling only, a straight line, or a flat line with both neurons dead
- **What helps:** more neurons (100 of 100 starts reached the good fit with 16), training from several random starts and keeping the one with the lowest validation loss, and the mini-batch noise and momentum that come up in a few slides

New York City daily electricity demand and temperature, 2,186 days. scikit-learn MLPRegressor with Adam and one hidden layer, with identical settings for every run except the random starting weights. The left panel is a sketch.

Backpropagation: computing the gradients

1. Forward pass
Run a batch of samples through the network and get predictions.

→

2. Loss
Compare the predictions with the targets. One number.

→

3. Backward pass
Compute the gradient of the loss for every weight, from the output layer back to the input.

→

4. Update
Move every weight a small step downhill.

Repeat with the next batch. One pass through all the training data is one epoch.

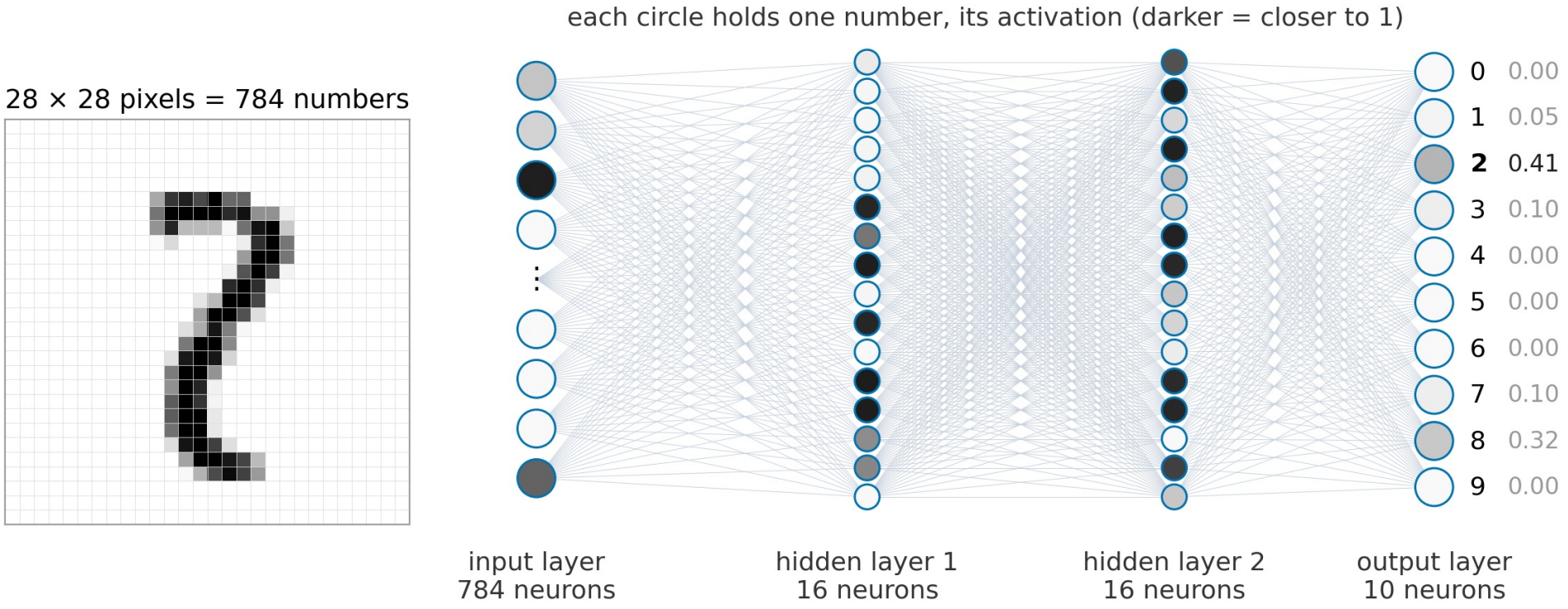
$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial w_3}, \ldots \right)$$

The gradient: one slope for every weight and bias, 13,002 numbers for the digit network. It points uphill, so the update moves every weight the other way.

- **The backward pass uses the chain rule.** The gradient for a weight in the first layer is a product of terms from every layer after it
- **You do not write it by hand.** TensorFlow and PyTorch record the forward pass and compute every gradient exactly, which is called automatic differentiation
- **Vanishing and exploding gradients:** a long product of small numbers goes to zero, and of large numbers blows up. ReLU, sensible starting weights, and scaled inputs keep deep networks trainable

For a visual walk-through on the digit network: 3Blue1Brown, What is backpropagation really doing? (3blue1brown.com/lessons/backpropagation).

A neural network for handwritten digits

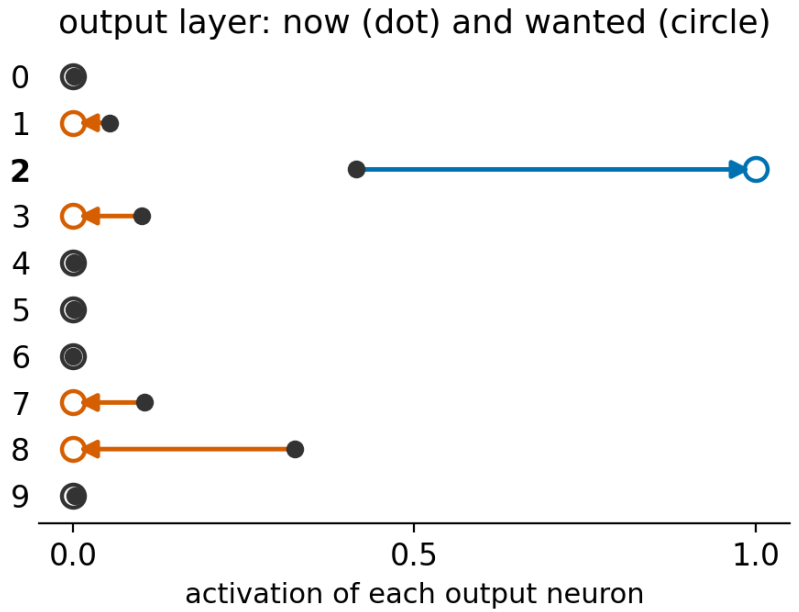


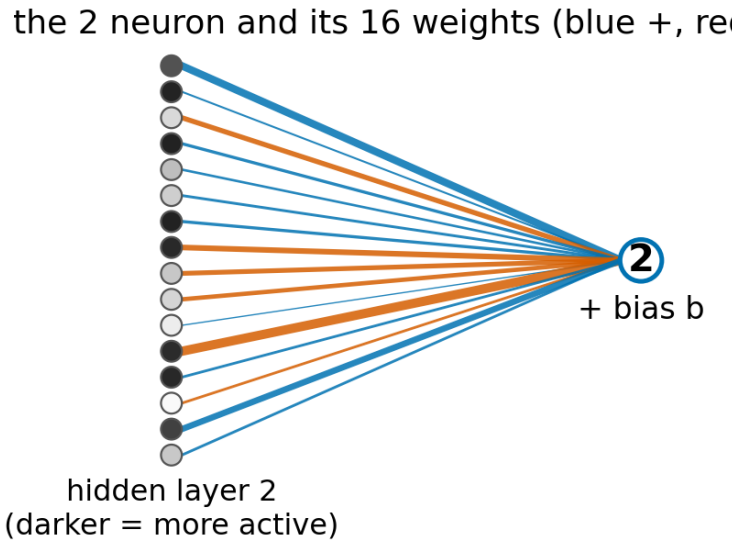
- **A neuron holds one number, its activation.** The 784 input neurons hold the pixel brightnesses. Each later neuron's number is computed from the layer before it
- **Layers:** 784 inputs, two hidden layers of 16 neurons, and 10 outputs, one per digit. The largest output is the network's answer
- **The whole network is a function:** 784 numbers in, 10 numbers out. Every line in the diagram is one weight

MNIST: 70,000 handwritten digits, 28 × 28 pixels. Following 3Blue1Brown's neural network lessons by Grant Sanderson, 3blue1brown.com/lessons/neural-networks. Network trained for these slides with scikit-learn.

Backpropagation: the nudges one example wants



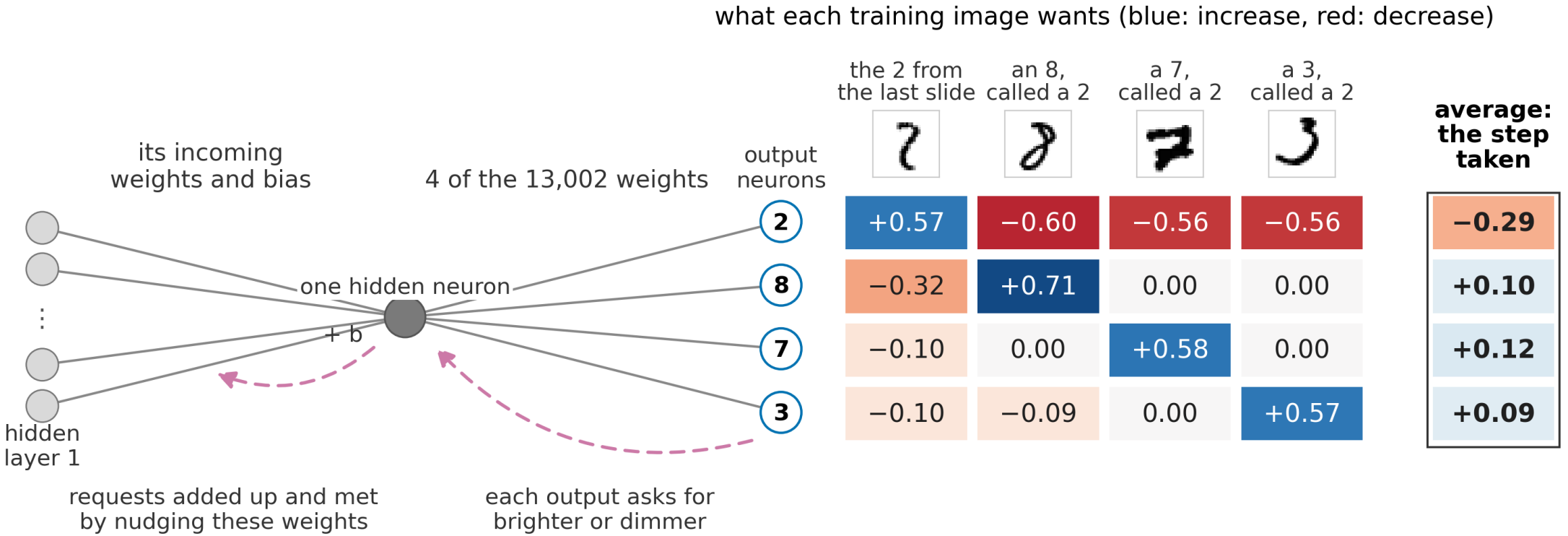




- **Each output should move toward its target:** the 2 neuron up from 0.41 toward 1, the others down toward 0, with bigger nudges for bigger misses
- **Three ways to raise the 2 neuron:** increase its bias, increase the weights from active neurons (change $\propto a_i$), or change the layer before (change $\propto w_i$)
- **Hebbian theory:** the biggest weight changes connect the most active neurons to the one that should change. This loosely echoes Hebb's 1949 theory of real neurons, often summed up as 'neurons that fire together wire together'
- **Then backward:** add up what all ten outputs want from hidden layer 2, and repeat the same reasoning one layer earlier

Following 3Blue1Brown, What is backpropagation really doing? (3blue1brown.com/lessons/backpropagation). Digit network trained for these slides with scikit-learn.

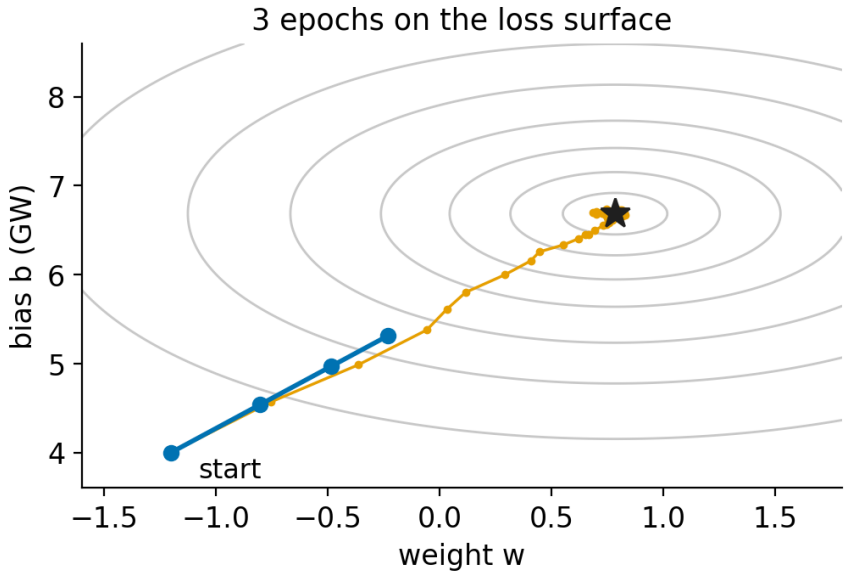
From each example's nudges to the gradient

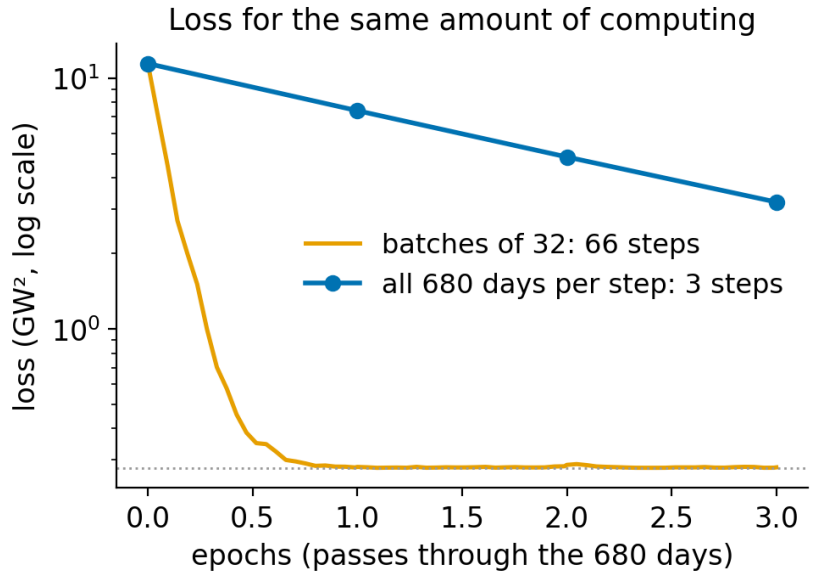


- **Each image wants its own nudges:** the 2 wants the weight into output 2 up, and the 8, 7, and 3 that are mistaken for 2s want it down
- **Gradient descent takes the average:** here that weight goes down (−0.29). Over all 60,000 images and all 13,002 weights and biases, the averages are the negative gradient
- **One layer back:** the outputs' requests for the hidden neuron are added up and met by nudging its own incoming weights and bias

Following 3Blue1Brown, What is backpropagation really doing? (3blue1brown.com/lessons/backpropagation). The trained digit network from the previous slide, on four training images it is unsure about.

Stochastic gradient descent: batches and epochs



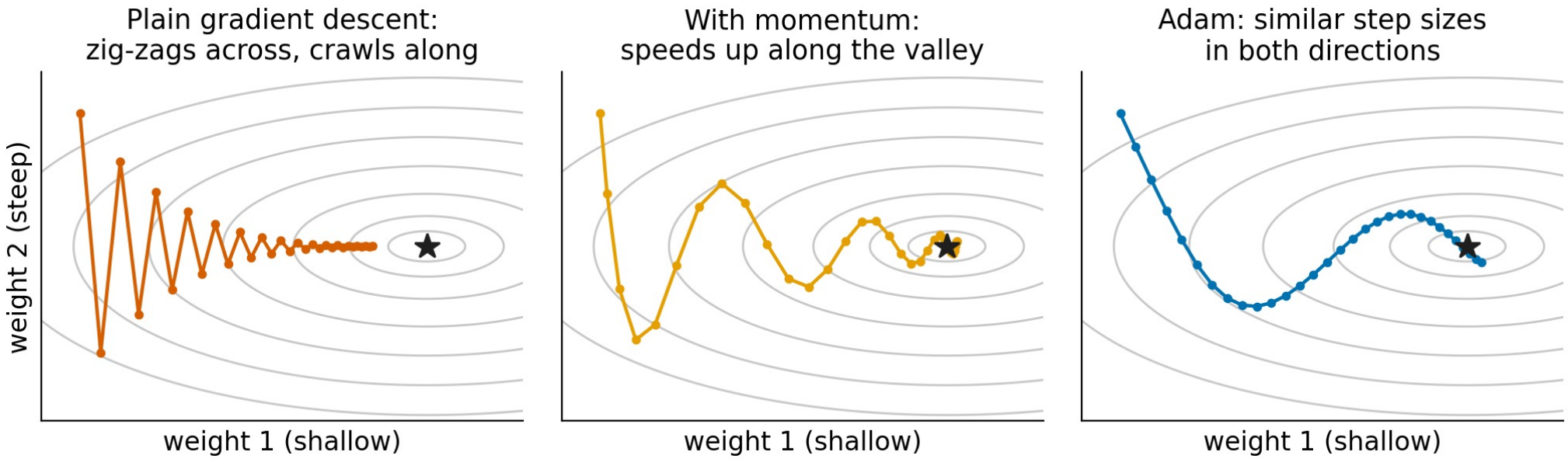


- **Mini-batch:** each step uses the average gradient over a small random batch, 32 by default in Keras, instead of all the training data
- **Epoch:** one pass through the training data: 22 steps for 680 days in batches of 32, but only one step if every step uses all the days
- **Stochastic gradient descent (SGD):** each step is noisy because its batch is a random sample, but there are many more steps. After 3 epochs SGD is at the minimum and full-batch gradient descent is still far away

```
model.fit(X_train, y_train, epochs=200, batch_size=32, validation_data=(X_valid, y_valid))
tf.random.set_seed(42)   # batches and starting weights are random: set a seed to repeat a run
```

Fitting demand = w × temperature + b to the 680 New York days above 20 °C, temperature standardized. Both methods start at the same point with learning rate 0.1.

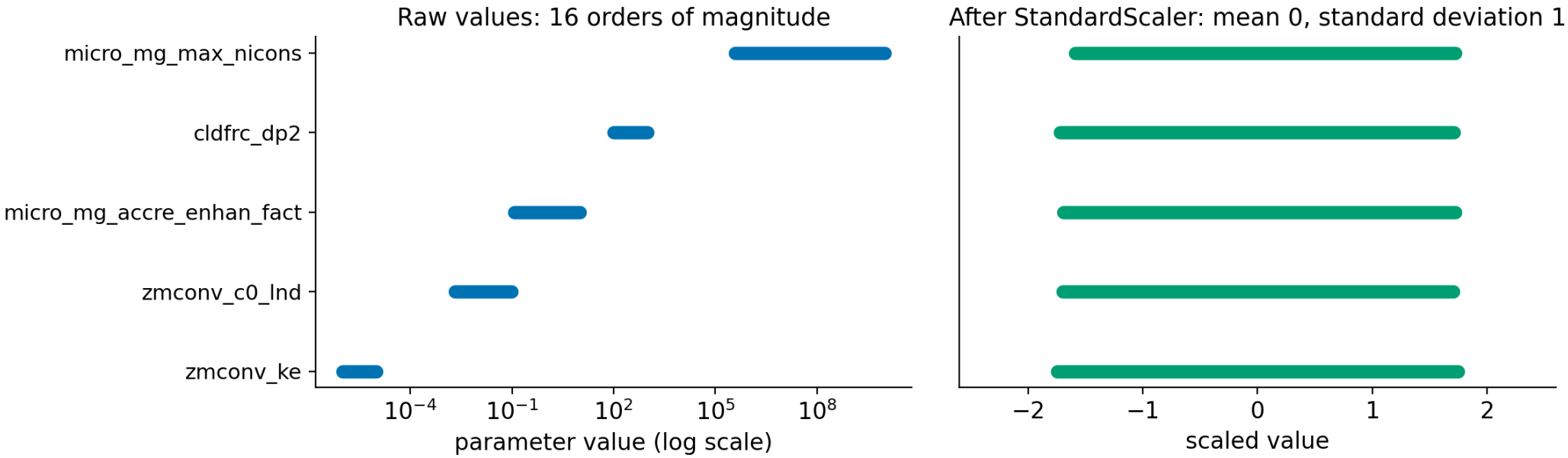
Optimizers: momentum and Adam



- **Plain gradient descent** moves every weight by the same learning rate times its gradient. In a valley that is steep one way and shallow the other, it bounces across the valley and crawls along it
- **Momentum** averages recent gradients. Steps that agree add up and speed along the valley, and steps that flip back and forth cancel
- **Adam** (Kingma and Ba, 2015) uses momentum and also divides each weight's step by the typical size of its recent gradients, so steep and shallow directions move at a similar pace
- **In practice:** start with Adam and its default learning rate of 0.001. It is optimizer="adam" in Keras and the default solver in scikit-learn

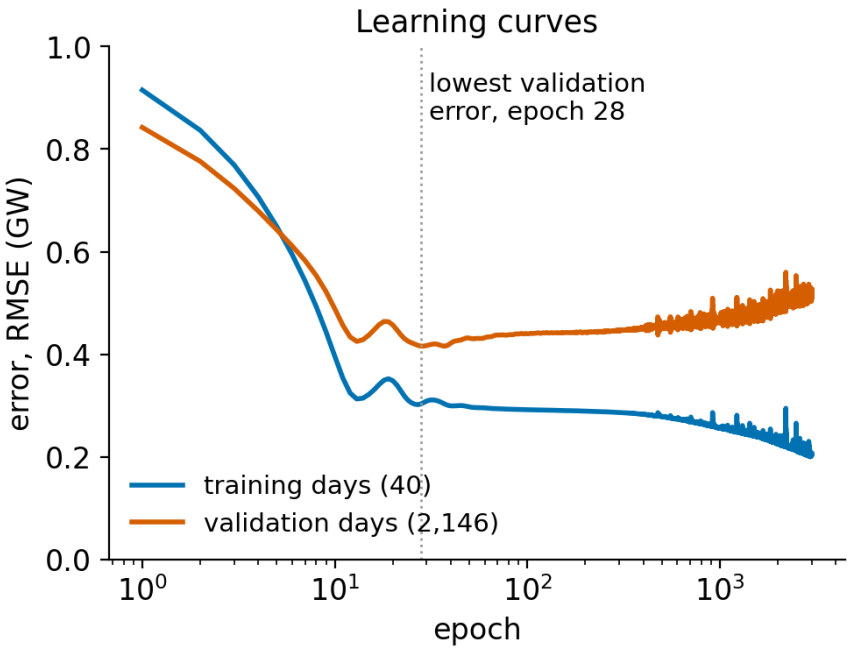
Sketch: the same made-up valley and 30 steps for each method from the same start, without batch noise. A real network's loss has thousands of directions, some steep and some shallow.

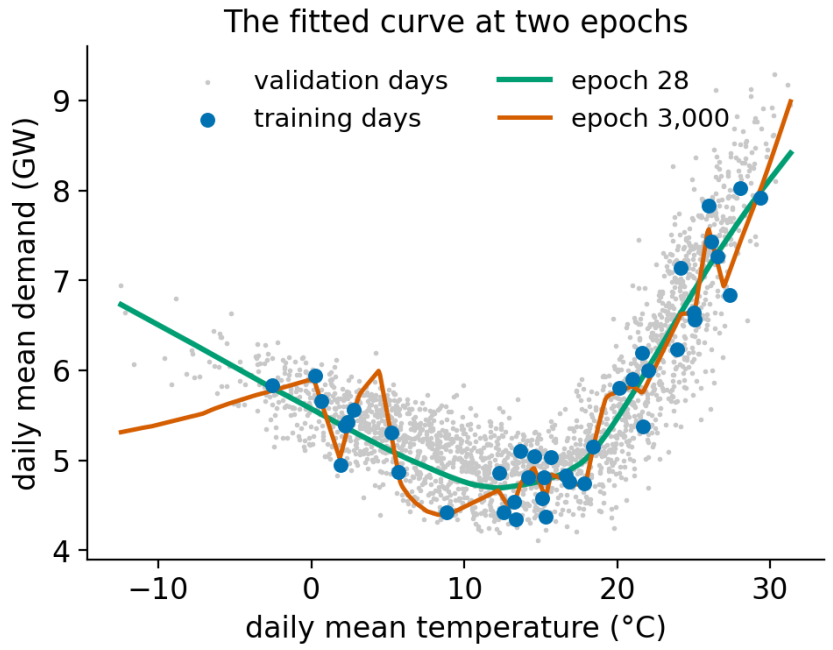
Scaling the inputs and the target



- **The 45 CAM6 parameters differ in size from 10^{-6} to 10^{10}.** Unscaled, the largest values dominate every weighted sum, and no single learning rate suits all the weights
- **Standardize every input** with StandardScaler, fit on the training set only, then applied to the validation and test sets (Week 6)
- **Standardize a regression target too,** and convert predictions back with inverse_transform so errors are in physical units such as W m^{-2} or mm/day
- **Networks need this more than random forests do.** A tree splits one feature at a time, so scale does not matter. A network adds the features together

Learning curves and overfitting

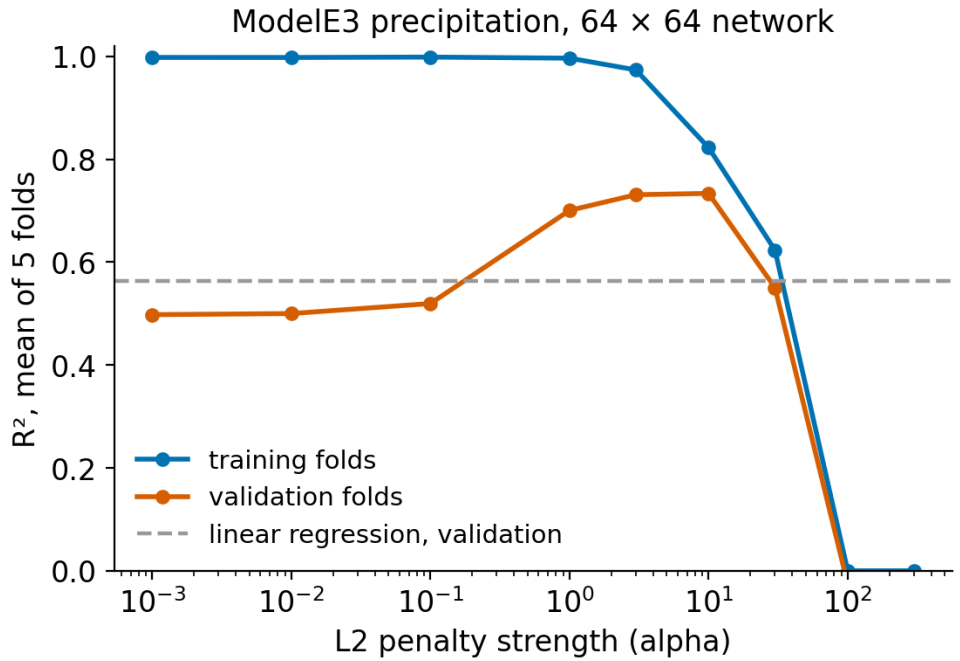




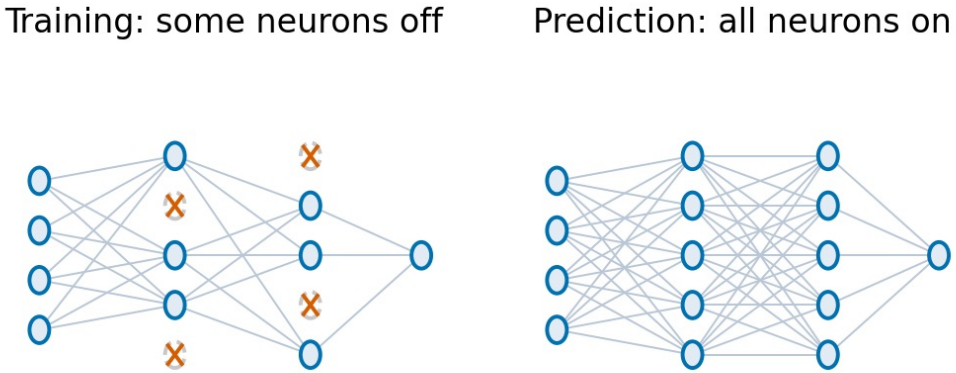
- **Learning curve:** the error on the training and validation sets after each epoch
- **Training error keeps falling.** With 40 days and about 17,000 parameters, the network can eventually pass through the training points
- **Validation error is lowest at epoch 28, 0.42 GW, and rises to 0.53 GW by epoch 3,000.** The later curve bends to reach individual days. That is overfitting
- **Early stopping:** keep the weights from the epoch with the lowest validation error

A deliberately small training set: 40 random New York days, two hidden layers of 128 neurons, no penalty, validated on the other 2,146 days.

Regularization: early stopping, L2, and dropout



L2 penalty: add the sum of squared weights to the loss, as in ridge regression. Validation R^2 rises from 0.50 to 0.73, then falls when the penalty is so strong the network predicts the mean.



Dropout: in each training step, every hidden neuron is switched off with some probability, often 0.2 to 0.5. The network cannot rely on any one neuron.

Early stopping: the previous slide. Cheap, and almost always worth using.

More data is the most reliable fix of all.

ModelE3 ensemble, global mean precipitation, 600 runs, 5-fold cross-validation. Linear regression scores 0.56. scikit-learn calls the L2 strength alpha.

Choosing the output layer and the loss

Task	Climate example	Output layer	Loss
Regression	Top-of-atmosphere net radiation from model parameters	1 neuron, no activation function	Mean squared error
Several targets at once	Radiation, cloud cover, and precipitation from the same parameters	1 neuron per target, no activation function	Mean squared error, targets standardized
Bounded target	Cloud fraction, between 0 and 1	1 neuron, sigmoid	Mean squared error
Yes or no	Will tomorrow exceed the ozone standard?	1 neuron, sigmoid (a probability)	Binary cross-entropy
One of several classes	Land cover type of a satellite pixel	k neurons, softmax (k probabilities that sum to 1)	Categorical cross-entropy

- **The hidden layers do not change with the task.** ReLU neurons, with sizes such as 32, 64, or 128. Only the output layer and the loss change
- **Cross-entropy** is the log loss from logistic regression. It is small when the network puts a high probability on what actually happened

Building and training a network in Keras

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.Input(shape=[45]),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1),        # regression: no activation
])
model.summary()                      # 7,169 parameters

model.compile(loss="mse", optimizer="adam", metrics=["mae"])

stop = tf.keras.callbacks.EarlyStopping(
    patience=10, restore_best_weights=True)

history = model.fit(Xs_train, ys_train, epochs=200,
                    validation_data=(Xs_valid, ys_valid),
                    callbacks=[stop])

y_pred = model.predict(Xs_test)      # still in scaled units
```

Sequential: a stack of layers, from input to output.

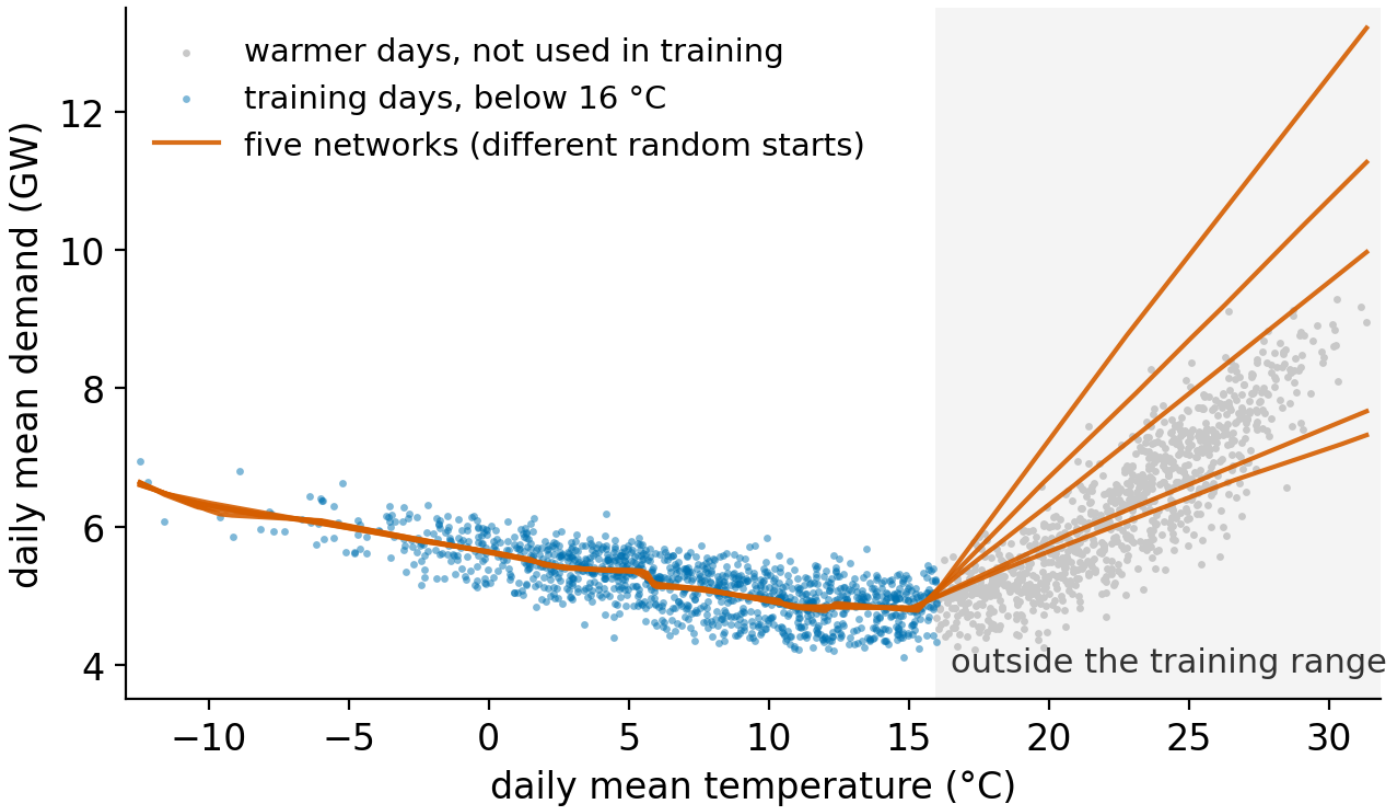
compile: the loss, the optimizer, and any metrics to print.

fit: runs the epochs. history.history holds the learning curves.

EarlyStopping: stops when the validation loss has not improved for patience epochs, and goes back to the best weights.

scikit-learn version:
MLPRegressor(hidden_layer_sizes=[64, 64]). Fine for small problems. Keras gives control over every layer, the loss, and training.

Neural networks do not extrapolate reliably



Inside the training range, five networks trained on the same cool days agree with each other and with the data.

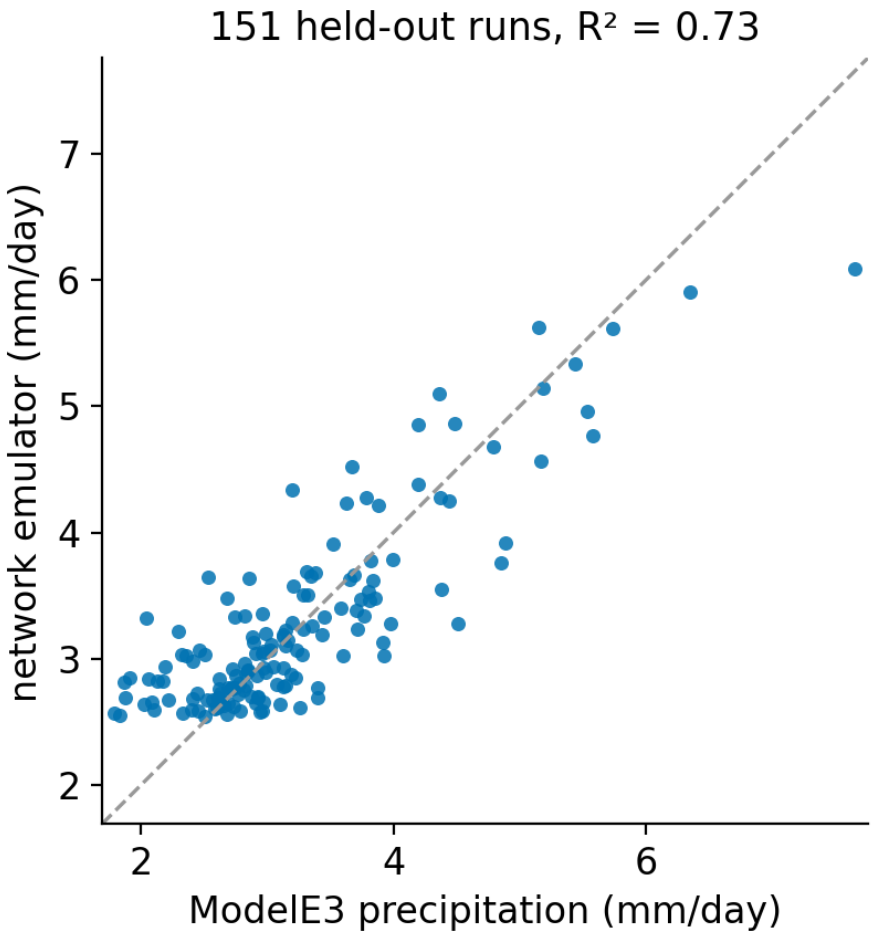
Outside it, they disagree. At 30 °C they predict 7.1 to 12.5 GW. Days above 28 °C average 8.4 GW.

The training days never needed air conditioning. Each network extends its last straight piece in its own direction.

Climate model example: a network trained on the present climate failed in a climate 4 K warmer. Trained on both climates, it worked in both (Rasp et al. 2018).

Check where new inputs fall relative to the training data. A warming climate moves inputs outside the range a network has seen.

Application: emulating a climate model ensemble



Perturbed parameter ensemble (PPE): run one climate model many times, each with a different setting of its uncertain parameters for clouds, convection, and turbulence. ModelE3: 751 runs. CAM6: 262 runs. 45 parameters each.

Emulator: a network from the 45 parameters to a global output. It predicts in milliseconds, so millions of parameter settings can be screened for those that match observations.

Result here: global mean precipitation for 151 runs the network never saw, R^2 = 0.73. Without the L2 penalty, 0.52. Linear regression, 0.59.

Simpler emulators can compete. On these two ensembles, Yang et al. (2025) found a network and a simpler additive model performed comparably, and the simpler one did better for CAM6.

ModelE3 ensemble from Yang et al. (2025), JAMES 17, e2024MS004766. Network with two hidden layers of 64 neurons, L2 strength chosen by cross-validation on the other 600 runs.

Steps for training a neural network

- **1. Split the data first:** training, validation, and test. Split by time or by place when nearby samples are related (Week 6)
- **2. Fit a simple baseline,** such as linear regression or a random forest. The network has to beat it to be worth using
- **3. Scale the inputs, and a regression target,** with statistics from the training set only
- **4. Start small:** one or two hidden layers of 32 to 64 ReLU neurons, Adam, and early stopping
- **5. Plot the learning curves.** A growing gap between training and validation error means overfitting. Add regularization, shrink the network, or get more data
- **6. Tune on the validation set,** then evaluate on the test set once
- **7. Check the range of new inputs.** Predictions outside the training data are not reliable

Extra

Reference slides: the equations, common hyperparameter values, and the code in one place.

Equations for a network with one hidden layer

Forward pass

$$a = \varphi(W_1 x + b_1), \qquad \hat{y} = W_2 a + b_2$$

x holds the d inputs and a the outputs of the m hidden neurons, so W_1 is m by d and W_2 is 1 by m. φ acts on each element of the weighted sum $z = W_1 x + b_1$, for example ReLU(z) = max(0, z).

Loss for regression: mean squared error

$$L = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

Gradient descent, for every weight

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

η is the learning rate. With mini-batches, L is computed on a batch of samples rather than the whole training set.

Backpropagation: the chain rule

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a} \cdot \frac{\partial a}{\partial W_1}$$

Each factor comes from one layer. The backward pass computes them from the output back, reusing each product for the layer before.

L2 penalty

$$L_{\text{total}} = L + \lambda \sum w^2$$

Softmax output and cross-entropy loss for k classes

$$p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}, \qquad L = -\sum_c y_c \log p_c$$

Each z is the weighted sum of one output neuron, before the activation function. y_c is 1 for the true class and 0 otherwise, so the loss is minus the log of the probability given to the true class.

Hyperparameters and common starting values

Hyperparameter	What it controls	Common starting value
Hidden layers	Depth	1 to 3 for tabular data
Neurons per layer	Width	32 to 256, often a power of 2
Hidden activation function	Where the bends come from	ReLU
Output activation function and loss	The type of target	See the output layer slide
Optimizer and learning rate	Step size	Adam, 0.001
Batch size	Samples per weight update	32
Epochs	Passes through the data	Large, with early stopping
L2 strength	Penalty on large weights	Try several orders of magnitude
Dropout rate	Share of neurons switched off	0.1 to 0.5

- **Tune on the validation set or with cross-validation, as in Week 6.** Change one thing at a time, and look at the learning curves after each change

Neural network code in scikit-learn and Keras

```
# scikit-learn
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

net = make_pipeline(
    StandardScaler(),
    MLPRegressor(hidden_layer_sizes=[64, 64],
                 alpha=1e-4,           # L2 strength
                 early_stopping=True,
                 max_iter=2000, random_state=0))
net.fit(X_train, y_train)
net.score(X_valid, y_valid)            # R²
net[-1].loss_curve_                    # loss per epoch
# MLPClassifier for classes, same arguments
```

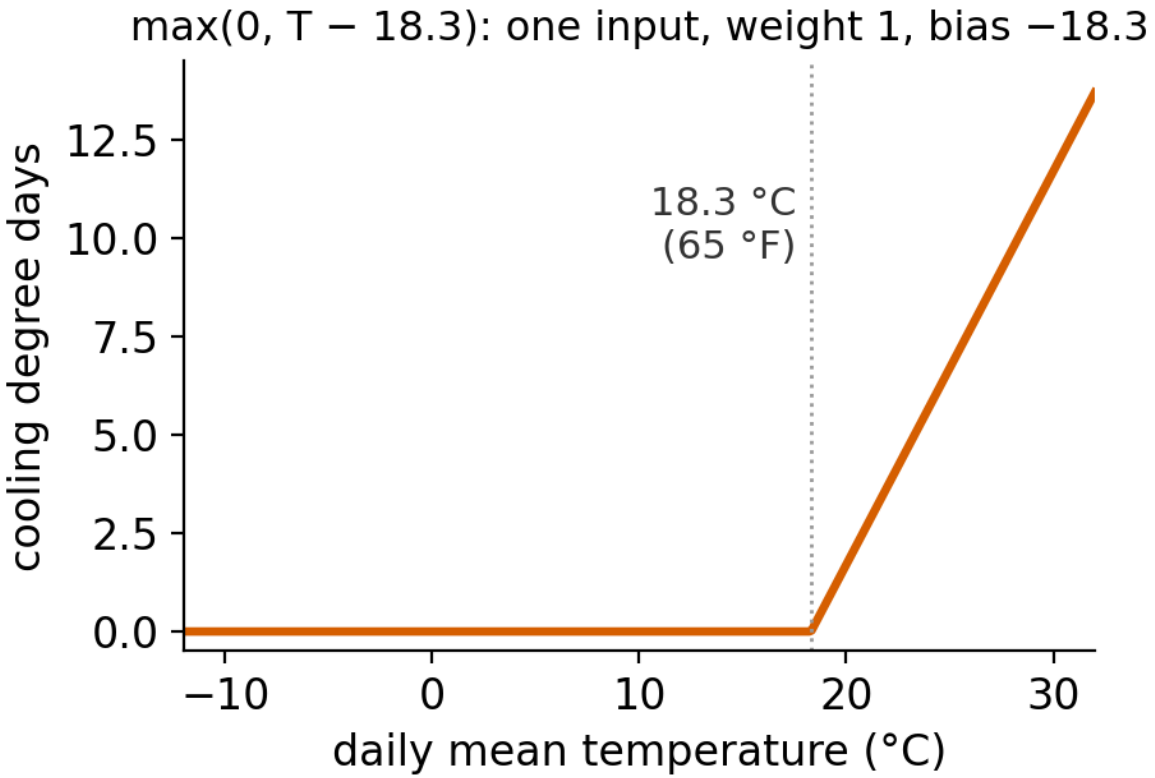
```
# Keras
from tensorflow.keras import layers, regularizers

layers.Dense(64, activation="relu",
             kernel_regularizer=regularizers.l2(1e-4))
layers.Dropout(0.2)                    # after a Dense layer
layers.Dense(1, activation="sigmoid")  # yes or no output
layers.Dense(k, activation="softmax")  # k classes

tf.keras.optimizers.Adam(learning_rate=1e-3)
loss="binary_crossentropy"             # with sigmoid
loss="sparse_categorical_crossentropy" # with softmax

h = history.history
h["loss"], h["val_loss"]               # learning curves
tf.random.set_seed(42)                 # repeatable runs
```

From digits to climate data: a ReLU neuron



$$z = 1 \cdot T - 18.3, \qquad \mathrm{CDD} = \max(0, z)$$

- **The same neuron works on any numbers.** Here it has one input, the daily mean temperature, instead of 784 pixels. A neuron is also called a unit, which is the term Keras uses
- **Cooling degree days are a ReLU neuron:** weight 1, bias −18.3 °C, and the activation function max(0, z). The output is 0 on cool days and rises 1 per °C on warm days
- **The 18.3 °C (65 °F) threshold is a convention** chosen by energy analysts. A network learns its own thresholds from data, as the two-neuron New York network shows

Counting the weights in a network

Layer	Weights	Biases	Parameters
Hidden layer 1	45 × 64 = 2,880	64	2,944
Hidden layer 2	64 × 64 = 4,096	64	4,160
Output layer	64 × 1 = 64	1	65
Total			7,169

Each neuron has one weight per input, plus a bias. A dense layer has (inputs × neurons) + neurons parameters.

Keras prints these counts with model.summary(). The tutorial's network has 7,169.

Digit network: 13,002 parameters
Trained on 60,000 images, about 4.6 images per parameter.

CAM6 emulator: 7,169 parameters
Trained on about 210 climate model runs, about 34 parameters per run. Linear regression on the same inputs has 46.

What follows from this
A network this size can memorize 210 runs. Training has to be watched on validation data, and the network kept in check.

Application: learning subgrid processes

The problem
A climate model grid column is about 200 km wide. Clouds and convection are much smaller. A parameterization estimates their effect on the column.

The training data
SPCAM runs a small cloud-resolving model, eight 4 km columns, inside every grid column. More accurate, and far too slow for routine climate runs.

The network
Learns SPCAM's answer from the state of the column, then runs inside the climate model in place of the parameterization.

94 inputs
Profiles of temperature, humidity, and wind. Surface pressure, sunlight, surface heat fluxes.

→

9 hidden layers of 256 neurons
About 570,000 weights.

→

65 outputs
Profiles of heating and moistening. Radiative fluxes. Precipitation.

- **Result:** the network ran stably inside the climate model for years of simulated time and reproduced SPCAM's climate. Its physics was about 20 times faster than SPCAM's
- **Limits:** an aquaplanet, an Earth covered by ocean, and it failed in a warmer climate it had not seen

Rasp, Pritchard, and Gentine (2018), Deep learning to represent subgrid processes in climate models, PNAS 115, 9684–9689. Host model: CAM3 with about 2° grid spacing.